

Anomaly Data Evolution — Two Classes, End to End

How a row of raw UART text becomes an ML result, for both fault classes, following the exact pipeline `run_all.ps1` runs.

The one idea to hold onto the whole way:

Memory leak → we **PREDICT** a number (seconds to crash). **Deadline miss** → we **DETECT** a state (is it wrong right now?). Everything below is downstream of that single difference.

Stage 0 — Raw UART text (identical format, different content)

The firmware prints `TELEM,uptime_ms,node,signal,seq,value` . Signals: `1 = heap_free` , `2 = heap_used` , `5 = loop_latency` .

Memory leak — `heap_free` falls, latency steady:

```
TELEM,200,1,1,0,8112      heap_free = 8112  (full)
TELEM,200,1,5,0,0        loop_latency = 0  (fine)
...
TELEM,111343,1,1,55,7976  heap_free dropping down
TELEM,120680,1,1,99,88    heap_free almost gone
```

Deadline miss — `heap_free` steady, latency climbs:

```
TELEM,200,1,1,0,8112      heap_free = 8112  (stays full)
TELEM,200,1,5,0,0        loop_latency = 0  (fine)
...
TELEM,59040,1,5,50,6      loop_latency rising up
TELEM,61221,1,5,99,81     loop_latency huge
```

Same shape of text — the fault lives in a *different signal*.

Stage 1 — `run_campaign.py` labels it → CSV (long format)

Parses each TELEM line, marks rows `faulty` after injection, and — **only for prediction classes** — computes `true_ttf` .

Memory leak (`ttf_sig = heap_free` → ttf gets filled):

timestamp	signal	value	label	true_ttf
200	heap_free	8112	normal	
111343	heap_free	7976	faulty	6.231
120680	heap_free	88	faulty	0.069

Deadline miss (`ttf_sig = None` → ttf always blank):

timestamp	signal	value	label	true_ttf
200	loop_latency	0	normal	
59040	loop_latency	6	faulty	
61221	loop_latency	81	faulty	

This is the fork, born right here: memory leak gets a target column (`true_ttf`); deadline miss never does. That one column decides everything downstream.

Stage 2 — `dataset.py` → `pivot_wide` (long → wide, one row per tick)

Signals become columns; `faulty` and `true_ttf` are merged back.

Memory leak:

episode	timestamp	heap_free	heap_used	loop_latency	faulty	true_ttf
0	200	8112	0	0	False	NaN
0	111300	7976	136	0	True	6.231
0	120700	88	8024	0	True	0.069

Deadline miss:

episode	timestamp	heap_free	heap_used	loop_latency	faulty	true_ttf
0	200	8112	0	0	False	NaN
0	59000	8112	0	6	True	NaN
0	61200	8112	0	81	True	NaN

Leak's `heap_free` moves; deadline's `loop_latency` moves; deadline's `true_ttf` is all `NaN`.

Stage 3 — `add_features` (add slopes)

Adds `heap_free_slope` and `loop_latency_slope` (change per second).

Memory leak — the *heap slope* is the informative one:

timestamp	heap_free	heap_free_slope	loop_latency	loop_latency_slope	faulty	true_ttf
200	8112	0.0	0	0.0	False	NaN
111300	7976	-1360.0	0	0.0	True	6.231
120700	88	0.0	0	0.0	True	0.069

Deadline miss — the *latency slope* is the informative one:

timestamp	heap_free	heap_free_slope	loop_latency	loop_latency_slope	faulty	true_ttf
200	8112	0.0	0	0.0	False	NaN
59000	8112	0.0	6	+50.0	True	NaN
61200	8112	0.0	81	+150.0	True	NaN

Up to here **the code is literally identical** — same functions, same frame. Now the two classes split into two different scripts.

The fork

Memory leak → `predictor.py` (PREDICT)

Stage 4a — `ramp_frame` : keep only the draining ramp. Filter `faulty & true_ttf > 0.1` → drops the flat baseline and the dead plateau. ~58 rows survive:

episode	timestamp	heap_free	heap_free_slope	true_ttf
0	111300	7976	-1360.0	6.231
0	111400	7840	-1360.0	6.125
...	...	...	...	...

Stage 5a — `ttf_split` : X = features, y = the number to predict.

```
Xtr = [[7976, -1360],      ytr = [6.231,      <- y is true_ttf (seconds)
       [7840, -1360]]      6.125]
```

Stage 6a — two predictors race, report MAE in seconds.

```
analytic: heap_free / -slope      -> [5.86, ...]
linreg:   learns ttf = a*heap_free -> [6.24, ...]

[analytic] MAE = 0.082 s    <- "off by 0.08 seconds on average"
[linreg ]  MAE = 0.000 s
```

Output = a time. "K1 will crash in X seconds."

Deadline miss → `baseline.py` (DETECT)

Stage 4b — no ramp filter. Keep everything, z-score it. Detection needs *normal* rows too (to learn what normal looks like), and no `true_ttf` exists to filter on anyway:

```
Xtr_normal = z-scored features, NORMAL rows only (train)
Xte         = z-scored features, normal + faulty (test)
yte         = [0,0,...,1,1] <- 0 = normal, 1 = faulty (a class, not a time)
```

Stage 5b — train on normal, score how "weird" each row is.

```
IsolationForest / Autoencoder, fit on normal only
-> anomaly score per test row (high = looks abnormal)
```

Stage 6b — report ROC-AUC (separability) + FP/hour.

```
[iso-forest] ROC-AUC = 0.504 <- can't tell normal from faulty (bad)
[denoise-AE] ROC-AUC = 1.000 <- separates them perfectly (good)
```

Output = a verdict. "This looks abnormal right now."

The whole thing in one glance

stage	Memory leak (PREDICT)	Deadline miss (DETECT)
raw signal that moves	<code>heap_free</code> down	<code>loop_latency</code> up
<code>true_ttf</code> column	filled (6.231 → 0)	empty (NaN)
shared: long → wide → slopes	same code	same code
the fork	filter to ramp	keep all + z-score
target <code>y</code>	seconds (a number)	class 0/1 (normal/faulty)
model	analytic ÷ / linreg	IsolationForest / autoencoder
script	<code>predictor.py</code>	<code>baseline.py</code>
metric	MAE (seconds)	ROC-AUC + FP/hour
answers	"crash in X sec"	"abnormal now?"

The mental hook: both classes walk *identical* rails from raw text to the slope-features table. Then the `true_ttf` column — present for the leak, absent for timing — sends them down two tracks: one **regresses a countdown**, the other **flags an outlier**.